

Implementation Plan

ApplyWizz Workday Dry-Run Automation Engine V1

1. File Name

implementation plan

2. Product Name

ApplyWizz Workday Dry-Run Automation Engine

3. Purpose of This Document

This document defines the exact build sequence for the ApplyWizz Workday Dry-Run Automation Engine V1.

The goal is to make sure the AI coding agent builds the system in the correct order without skipping foundation layers.

This plan should be followed phase by phase.

The AI agent must not jump ahead to advanced automation before the database, authentication, candidate setup, resume upload, and job queue are working.

4. Core Build Principle

Build in this order:

```
Foundation
↓
Database
↓
Auth
↓
Candidate
↓
Resume
```

↓
Job Links
↓
Worker
↓
Workday Browser
↓
Zoho OTP
↓
Question Extraction
↓
Answer Mapping
↓
Manual Review
↓
Dry-Run Summary
↓
Human Approval
↓
Controlled Submit
↓
Testing
↓
Deployment

Do not build advanced agents, auto-submit, or multiple portals in V1.

5. V1 Build Rule

V1 must focus only on:

One candidate
+
One resume
+
One Zoho email
+
One Workday job link
+
Dry-run before submit
+
Manual review if unsure

V1 must not include:

```
Greenhouse
Lever
iCIMS
Multiple resumes
Full auto-submit without approval
Browser Use
Agent Ship
SkillClaw
AgentHub
Alibaba Page Agent
CAPTCHA bypass
Anti-detect browser tools
```

Phase 1 — Project Setup

Goal

Create the technical foundation of the project.

This phase sets up the monorepo, frontend app, worker app, shared packages, TypeScript, environment files, and basic developer tooling.

Build

Create monorepo structure:

```
applywizz-workday-automation/
|
├─ apps/
|   └─ mobile-web/
|       └─ worker/
|
├─ packages/
|   └─ shared/
|       └─ database/
|
├─ docs/
|
├─ .env.example
└─ package.json
```

```
|— pnpm-workspace.yaml
|— README.md
```

Set up:

- pnpm workspace
- Expo React Native app
- Expo Router
- React Native Web
- TypeScript
- NativeWind
- worker Node.js TypeScript app
- shared package
- database package
- ESLint
- Prettier
- Vitest
- basic README
- `.env.example`

Required Environment Variables

Add placeholders only, not real values:

```
EXPO_PUBLIC_SUPABASE_URL
EXPO_PUBLIC_SUPABASE_ANON_KEY
EXPO_PUBLIC_APP_ENV

SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY

OPENAI_API_KEY
OPENAI_MODEL

SCRAPEGRAPHAI_API_KEY
SCRAPEGRAPHAI_ENDPOINT

ZOHO_CLIENT_ID
ZOHO_CLIENT_SECRET
ZOHO_REDIRECT_URI

ENCRYPTION_KEY
WORKER_CONCURRENCY
WORKER_POLL_INTERVAL_MS
```

```
WORKER_MAX_RETRIES
APP_ENV
LOG_LEVEL
```

Done Criteria

Phase 1 is complete when:

- monorepo exists
- Expo app starts locally
- worker app compiles
- shared package is importable
- TypeScript works
- lint command works
- test command works
- `.env.example` exists
- README explains how to run local development
- git status is clean

Phase 2 — Database Schema and Migrations

Goal

Create the Supabase database foundation before building frontend features.

The AI agent must not build UI features that depend on missing tables.

Build

Create migrations for these tables:

```
users
candidates
candidate_resumes
candidate_answers
question_bank
zoho_mailboxes
workday_accounts
job_links
application_runs
```

```
run_steps
extracted_questions
mapped_answers
manual_review_items
otp_logs
screenshots
automation_logs
system_settings
audit_logs
```

Create:

- UUID primary keys
- foreign keys
- timestamps
- updated_at triggers
- status check constraints
- indexes
- partial unique index for one active resume per candidate
- unique candidate + normalized job URL constraint
- RLS enabled on all user-facing tables
- basic RLS policies
- storage buckets

Required private storage buckets:

```
candidate-resumes
run-screenshots
playwright-traces
html-captures
```

Create database functions:

```
claim_next_application_run()
calculate_run_readiness()
```

Seed system settings:

```
dry_run_required = true
human_approval_required = true
auto_submit_enabled = false
max_worker_concurrency = 1
```

```
otp_timeout_seconds = 120
default_run_mode = dry_run
```

Done Criteria

Phase 2 is complete when:

- all migrations run successfully on local Supabase
- all tables exist
- all indexes exist
- all storage buckets exist and are private
- RLS is enabled
- seed settings exist
- `claim_next_application_run()` works
- `calculate_run_readiness()` works
- database types are generated
- migration rollback plan is documented
- tests verify critical constraints
- git status is clean

Phase 3 — Authentication Flow

Goal

Allow ApplyWizz operators to log in securely and access protected pages.

Build

Use:

```
Supabase Auth
Email + Password
```

Create frontend screens:

```
/login
/dashboard
```

Create app user profile flow:

- after Supabase Auth login, load user from `users` table
- protect all dashboard routes
- redirect unauthenticated users to login
- show logged-in user in header
- support logout

Roles:

```
admin
operator
viewer
```

Implement role helper functions in shared package.

Done Criteria

Phase 3 is complete when:

- user can log in
 - user can log out
 - unauthenticated user is redirected to login
 - logged-in user lands on dashboard
 - user role is loaded from `users` table
 - protected routes work
 - viewer/operator/admin role helpers work
 - invalid login shows clear error
 - loading state works
 - tests pass
 - git status is clean
-

Phase 4 — Core Layout and Navigation

Goal

Build the main application shell before individual feature pages.

Build

Create logged-in app layout:

- left sidebar
- top header
- main content area
- environment badge
- worker status placeholder
- user menu

Sidebar items:

Dashboard
Candidates
Job Links
Runs
Manual Review
Answer Bank
Settings

Create placeholder routes for all main pages.

Apply UI/UX direction:

- dark mode first
- Linear/Vercel style
- cards
- tables
- status badges
- clean spacing
- responsive foundations

Done Criteria

Phase 4 is complete when:

- sidebar navigation works
- all main pages exist
- active sidebar item is highlighted
- header appears on all logged-in pages
- layout works on desktop
- basic mobile adaptation does not break
- dark theme is applied consistently

- no broken routes
- git status is clean

Phase 5 — Candidate Management

Goal

Allow operators to create and manage candidate profiles.

Build

Create:

```
/candidates  
/candidates/new  
/candidates/[id]
```

Candidate list page:

- search
- filter by status
- table/card list
- empty state
- add candidate button

Create candidate page:

- full name
- email
- phone
- location
- target role
- years of experience
- LinkedIn URL
- GitHub URL
- portfolio URL
- current company
- expected salary
- availability
- relocation preference
- sponsorship requirement
- work authorization

- notes

Candidate detail page:

- profile summary
 - edit profile action
 - resume section placeholder
 - Zoho section placeholder
 - job links section placeholder
 - runs section placeholder
 - manual review section placeholder
-

Done Criteria

Phase 5 is complete when:

- operator can create candidate
 - operator can edit candidate
 - operator can view candidate detail
 - candidate list loads from Supabase
 - empty state works
 - search works
 - required validation works
 - viewer cannot edit if role restrictions are enabled
 - audit log records candidate creation/update
 - tests pass
 - git status is clean
-

Phase 6 — Resume Upload

Goal

Allow one active resume per candidate.

Build

Implement private resume upload to Supabase Storage.

Rules:

- V1 supports only one active resume per candidate
- resume files go to `candidate-resumes` bucket

- path format:

```
candidate-resumes/{candidate_id}/{resume_id}/{file_name}
```

Allowed file types:

```
PDF  
DOC  
DOCX
```

Build:

- upload resume modal
- replace resume confirmation modal
- resume metadata save
- signed URL preview/download
- active resume display
- resume replace flow

Done Criteria

Phase 6 is complete when:

- operator can upload resume
- resume is stored in private bucket
- resume metadata is saved
- only one active resume exists per candidate
- replacing resume deactivates old active resume
- signed URL access works
- invalid file type is rejected
- missing resume state displays clearly
- audit log records resume upload/replace
- tests pass
- git status is clean

Phase 7 — Zoho Mailbox Setup

Goal

Prepare candidate Zoho email connection for Workday OTP reading.

Build

Create Zoho mailbox section on candidate detail page.

States:

```
not_connected
connected
expired
failed
revoked
```

Build backend/worker-safe flow for:

- storing mailbox metadata
- starting Zoho OAuth
- handling OAuth callback
- encrypting access token
- encrypting refresh token
- testing OTP email access
- showing connection status

Important privacy rule:

The system should search only OTP-related emails. It should not expose inbox contents in the UI.

Done Criteria

Phase 7 is complete when:

- operator can start Zoho connection
 - OAuth callback stores encrypted tokens
 - connection status is shown
 - test OTP access button works or returns a controlled failure
 - tokens are never exposed to frontend
 - viewer cannot access sensitive data
 - audit log records Zoho connection
 - tests pass
 - git status is clean
-

Phase 8 — Job Link Management

Goal

Allow operators to add Workday job links and prepare them for dry-run automation.

Build

Create:

```
/job-links  
Candidate Detail → Job Links section
```

Features:

- add one Workday link
- bulk paste links
- validate Workday URL
- normalize URL
- detect duplicates
- save company name if provided
- save job title if provided
- assign candidate
- show job status
- skip job
- retry failed job placeholder

Statuses:

```
queued  
running  
opened  
login_required  
otp_required  
logged_in  
resume_uploaded  
questionnaire_reached  
questions_extracted  
answers_mapped  
manual_review_required  
dry_run_complete  
approved_for_submit  
submitted
```

failed
duplicate
skipped

Done Criteria

Phase 8 is complete when:

- operator can add one Workday link
 - operator can bulk paste links
 - invalid links are rejected
 - duplicates are detected
 - links appear under candidate detail
 - global job links page works
 - filters by status work
 - audit log records job link creation
 - tests pass
 - git status is clean
-

Phase 9 — Application Run Creation

Goal

Create dry-run records from job links before the worker automation begins.

Build

Add **Start Dry Run** action.

Before creating run, validate:

- candidate exists
- candidate has one active resume
- Zoho mailbox is connected
- job link is valid
- job is not already running

When clicked:

```
Create application_run
Set mode = dry_run
Set status = queued
Update job_link status = queued
Redirect to Run Detail Page
```

Create:

```
/runs
/runs/[id]
```

Run detail should initially show:

- run summary
- current status
- step timeline placeholder
- screenshots placeholder
- extracted questions placeholder
- mapped answers placeholder
- manual review placeholder

Done Criteria

Phase 9 is complete when:

- Start Dry Run button works
- missing resume blocks run creation
- missing Zoho blocks run creation
- queued application_run is created
- user redirects to run detail page
- job link status updates
- run appears in Runs page
- audit log records dry-run start
- tests pass
- git status is clean

Phase 10 — Worker Foundation

Goal

Build the backend worker that safely claims queued runs and writes progress.

Build

In `apps/worker`, implement:

- Supabase service role client
- environment validation
- queue polling
- `claim_next_application_run()` usage
- run status updates
- run step creation
- automation logs
- screenshot upload helper
- HTML capture upload helper
- graceful shutdown
- retry skeleton
- worker concurrency config

Worker must use:

SUPABASE_SERVICE_ROLE_KEY

Frontend must never access service role key.

Done Criteria

Phase 10 is complete when:

- worker starts locally
 - worker connects to Supabase
 - worker claims queued run safely
 - worker updates run status from queued to starting
 - worker writes run steps
 - worker writes automation logs
 - worker does not double-process same run
 - worker handles no queued jobs gracefully
 - tests pass
 - git status is clean
-

Phase 11 — Playwright Browser Base

Goal

Open Workday links with Playwright and record evidence.

Build

Add Playwright to worker.

Implement:

- browser launch
- page creation
- navigate to Workday URL
- page load timeout
- current URL capture
- page title capture
- screenshot capture
- run step updates
- browser close cleanup
- error handling

Status flow:

```
starting
↓
opening_job_link
↓
detecting_page_state
```

Done Criteria

Phase 11 is complete when:

- worker opens a Workday job link
- screenshot is saved to private bucket
- screenshot metadata is saved
- run step timeline updates
- current URL is logged
- page title is logged

- failed page load is handled
 - Playwright closes cleanly
 - run detail displays screenshot
 - tests pass
 - git status is clean
-

Phase 12 — Workday Page State Detection

Goal

Detect where the user is inside the Workday application flow.

Build

Implement page state detection for:

```
job_description_page
apply_button_visible
login_page
create_account_page
otp_required_page
resume_upload_page
questionnaire_page
review_page
confirmation_page
unsupported_page
```

Use:

- visible text
- URL patterns
- buttons
- form labels
- accessibility selectors
- screenshots for debugging

Do not fill anything yet.

Done Criteria

Phase 12 is complete when:

- worker can identify major Workday page states
 - unsupported page is handled safely
 - each state writes a run step
 - screenshots are saved for state transitions
 - state detection logic has unit tests where possible
 - no auto-submit is possible
 - git status is clean
-

Phase 13 — Workday Login and Account Creation

Goal

Allow the worker to log in or create Workday accounts for a candidate.

Build

Implement:

- tenant key detection
- workday account lookup
- account creation flow
- login flow
- password generation
- password encryption
- account status updates
- account already exists handling
- login failed handling

Do not build password reset in V1 unless required.

Done Criteria

Phase 13 is complete when:

- worker detects tenant key
- worker can create account on supported tenant
- worker can log in with existing credentials

- encrypted password is stored
 - account status updates
 - failures are logged clearly
 - screenshots are saved
 - no password is exposed in UI
 - tests pass
 - git status is clean
-

Phase 14 — Zoho OTP Automation

Goal

Read Workday OTP from the candidate's Zoho mailbox and continue login/account creation.

Build

Implement worker functions:

- refresh Zoho token if needed
- search OTP-related Workday emails
- extract OTP code
- mask OTP in logs
- input OTP into Workday
- mark OTP success/failure
- timeout handling
- retry OTP read within configured timeout

Do not store full email content.

Do not expose OTP in frontend.

Done Criteria

Phase 14 is complete when:

- worker can search Zoho for OTP email
- OTP parser works
- OTP timeout is handled
- OTP logs are created
- OTP is masked in logs
- worker can enter OTP into Workday
- no full email content is stored

- tests pass
 - git status is clean
-

Phase 15 — Resume Upload to Workday

Goal

Upload the candidate's active resume into Workday.

Build

Implement:

- fetch active resume metadata
- create signed URL or worker-accessible download
- download resume to worker temp storage
- detect Workday upload field
- upload file with Playwright
- verify file appears on page
- continue to next step
- cleanup temp file

Status flow:

```
uploading_resume
↓
resume_uploaded
↓
reaching_questionnaire
```

Done Criteria

Phase 15 is complete when:

- worker fetches candidate active resume
- worker uploads resume to Workday
- upload success is verified
- screenshot after upload is saved
- temp files are cleaned up
- failure is logged clearly

- run status updates correctly
 - tests pass
 - git status is clean
-

Phase 16 — Questionnaire Page Detection

Goal

Reach the Workday questionnaire page and stop before answering.

Build

Implement:

- detect questionnaire page
- identify forms
- identify visible sections
- capture page HTML
- capture accessibility-relevant labels
- save screenshot
- save HTML capture to `html-captures`
- update status to `questionnaire_reached`

Important:

This phase does not map or fill answers yet.

Done Criteria

Phase 16 is complete when:

- worker reaches questionnaire page
 - screenshot is saved
 - HTML capture is saved
 - run status becomes `questionnaire_reached`
 - run detail shows evidence
 - unsupported flows fail safely
 - tests pass
 - git status is clean
-

Phase 17 — HTML Compression and Question Extraction

Goal

Extract structured questions from the Workday questionnaire page.

Build

Implement HTML compression:

Remove:

```
scripts
styles
layout noise
tracking code
irrelevant navigation
```

Keep:

```
labels
input names
aria labels
select options
radio labels
checkbox labels
visible text
required markers
section headings
```

Send compressed HTML to ScrapeGraphAI.

Expected extracted question object:

```
question_text
normalized_question
field_type
options
is_required
```



```
is_visible
section_name
page_step
field_selector
extraction_confidence
risk_level
```

Save results to `extracted_questions`.

Done Criteria

Phase 17 is complete when:

- HTML compression works
 - ScrapeGraphAI integration works
 - questions are extracted
 - extracted questions are saved
 - field types are identified
 - options are captured
 - required status is captured where possible
 - extraction failure is handled
 - run detail displays extracted questions
 - tests pass
 - git status is clean
-

Phase 18 — Answer Bank and Risk Classification

Goal

Prepare safe answer mapping before filling anything.

Build

Create Answer Bank UI:

```
/answer-bank
Candidate Detail → Answer Bank section
```

Support:

- candidate-specific answers
- global question bank
- question key
- normalized question
- answer value
- answer type
- risk level
- approval status
- category
- usage count

Risk levels:

low
medium
high
unknown

Rules:

- low risk can be auto-filled if answer exists
- medium risk can be auto-filled only with high confidence and approved answer
- high risk always requires manual review
- unknown always requires manual review

Done Criteria

Phase 18 is complete when:

- answer bank page exists
 - candidate answers can be created
 - global question bank entries can be viewed
 - risk level is stored
 - approval status is stored
 - high-risk answers cannot be auto-approved by AI
 - tests pass
 - git status is clean
-

Phase 19 — OpenAI Answer Mapping

Goal

Map extracted Workday questions to approved candidate/global answers.

Build

Implement answer mapping flow:

```
Extracted Question
↓
Normalize
↓
Check candidate_answers
↓
Check question_bank
↓
Use OpenAI semantic matching if needed
↓
Return confidence score
↓
Classify risk
↓
Create mapped_answers
```

Rules:

- OpenAI must not invent answers
- OpenAI can only match question to known approved data
- high-risk questions go to manual review
- unknown questions go to manual review
- low confidence goes to manual review

Create mapping statuses:

```
pending
mapped
needs_review
approved
rejected
```

filled
skipped

Done Criteria

Phase 19 is complete when:

- extracted questions map to candidate answers
- extracted questions map to question bank entries
- confidence score is stored
- source type is stored
- risk level is stored
- manual review is created when needed
- OpenAI output is validated with Zod
- no answer is invented
- tests pass
- git status is clean

Phase 20 — Manual Review System

Goal

Allow humans to review risky, unknown, or low-confidence answers.

Build

Create:

```
/manual-review  
Manual Review drawer  
Run Detail → review section
```

Review item should show:

- candidate
- company
- job title
- question
- field type
- options

- suggested answer
- confidence score
- risk level
- reason for review
- screenshot preview

Actions:

Approve Suggested Answer
Edit Answer
Mark Unsafe
Skip Question
Save to Answer Bank
Continue Run

Done Criteria

Phase 20 is complete when:

- manual review page lists open items
- drawer opens with full details
- operator can approve answer
- operator can edit answer
- operator can mark unsafe
- operator can skip question
- approved answers update mapped_answers
- resolved review items are saved
- audit logs are created
- tests pass
- git status is clean

Phase 21 — Safe Form Filling

Goal

Fill only safe and approved answers into Workday.

Build

Implement Playwright filling for:

```
text
number
email
phone
date
dropdown
radio
checkbox
multi_select
yes_no
address
legal_acknowledgment
```

Rules:

- fill only mapped and approved answers
- do not fill high-risk unresolved answers
- do not guess missing answers
- save screenshot after filling
- update mapped_answers status to filled
- update run counts

Done Criteria

Phase 21 is complete when:

- worker fills low-risk approved answers
- dropdowns work
- radio buttons work
- checkboxes work
- text inputs work
- unresolved risky questions are skipped and sent to review
- filled answers are recorded
- screenshot after fill is saved
- tests pass
- git status is clean

Phase 22 — Conditional Question Handling

Goal

Detect new Workday questions that appear after answering visible questions.

Build

Implement loop:

```
Capture questions before fill
↓
Fill safe answers
↓
Wait for page changes
↓
Capture questions again
↓
Diff before vs after
↓
Extract new questions
↓
Map new questions
↓
Fill if safe
↓
Manual review if unknown/risky
```

Set maximum loop count to prevent infinite loops.

Recommended:

```
MAX_CONDITIONAL_LOOPS = 5
```

Done Criteria

Phase 22 is complete when:

- worker detects newly appearing questions
- new questions are saved
- new questions are mapped
- safe new answers can be filled
- risky new questions go to review
- loop stops safely
- infinite loops are prevented
- tests pass
- git status is clean

Phase 23 — Dry-Run Completion and Readiness Score

Goal

Stop before final submission and show a human-readable summary.

Build

Create Dry-Run Summary section/page.

Show:

- candidate
- company
- job title
- Workday URL
- resume uploaded
- account used
- total questions found
- total answers mapped
- total answers filled
- total manual review items
- total high-risk items
- unresolved questions
- screenshots
- readiness score

Readiness score:

```
ready  
needs_review  
blocked  
failed
```

Use `calculate_run_readiness()`.

Done Criteria

Phase 23 is complete when:

- dry-run stops before final submit
 - readiness score is calculated
 - summary is displayed
 - unresolved review items show Needs Review
 - failed critical steps show Blocked or Failed
 - ready run shows approval option
 - no final submit happens automatically
 - tests pass
 - git status is clean
-

Phase 24 — Final Human Approval

Goal

Allow approved users to approve final submission.

Build

Only admin can approve final submit in V1.

Create approval action:

Approve Final Submit

When clicked:

```
Check readiness_score = ready
Check no unresolved required manual review
Check user role = admin
Set approved_by
Set approved_at
Set status = approved_for_submit
Write audit log
```

Show confirmation modal.

Done Criteria

Phase 24 is complete when:

- only admin can approve final submit
 - approval requires readiness_score = ready
 - unresolved manual review blocks approval
 - approval updates run status
 - approval writes audit log
 - UI clearly shows who approved and when
 - tests pass
 - git status is clean
-

Phase 25 — Controlled Submit

Goal

Submit the Workday application only after human approval.

Build

Implement worker-controlled submit.

Submit may happen only if:

```
application_runs.status = approved_for_submit  
approved_by is not null  
approved_at is not null  
readiness_score = ready
```

Worker action:

```
Resume browser/session if possible  
Click final submit  
Capture confirmation screenshot  
Set status = submitted  
Set submitted_at  
Update job_link status = submitted  
Write audit log
```

If browser session cannot resume, show blocked status and ask operator to manually submit or retry.

Done Criteria

Phase 25 is complete when:

- submit endpoint rejects unapproved runs
 - submit works only after approval
 - confirmation screenshot is saved
 - run status becomes submitted
 - job link status becomes submitted
 - audit log is written
 - failed submit is handled safely
 - tests pass
 - git status is clean
-

Phase 26 — Dashboard Metrics

Goal

Give operators a simple command center view.

Build

Dashboard cards:

```
Total Candidates
Queued Jobs
Running Jobs
Manual Review Items
Dry-Runs Complete
Failed Jobs
Submitted Applications
Worker Health
```

Sections:

- today's activity
- jobs needing attention
- recent runs

- recent failures
 - worker status
-

Done Criteria

Phase 26 is complete when:

- dashboard loads real counts
 - cards link to filtered pages
 - recent runs show latest data
 - manual review card works
 - failed jobs card works
 - empty state works
 - loading state works
 - tests pass
 - git status is clean
-

Phase 27 — UI Polish and Responsive Design

Goal

Apply consistent UI/UX across the product.

Build

Polish:

- dark mode consistency
- spacing
- typography
- buttons
- cards
- tables
- badges
- modals
- drawers
- empty states
- loading states
- error states
- mobile-friendly structure

Follow **ui ux** document.

Important pages:

Dashboard
Candidates
Candidate Detail
Job Links
Runs
Run Detail
Manual Review
Answer Bank
Settings
Dry-Run Summary

Done Criteria

Phase 27 is complete when:

- UI is consistent across all pages
- status badges use correct colors
- risk badges use correct colors
- dangerous actions require confirmation
- layout works on desktop
- layout does not break on tablet/mobile widths
- empty states are helpful
- errors are understandable
- no placeholder UI remains
- git status is clean

Phase 28 — Error Handling and Edge Cases

Goal

Make the product safe and understandable when things go wrong.

Build

Handle:

```
invalid Workday link
duplicate job link
missing resume
Zoho not connected
Zoho token expired
OTP not found
OTP expired
Workday timeout
apply button not found
login failed
account already exists
resume upload failed
question extraction failed
answer mapping failed
unsupported field type
risky question detected
unknown conditional question
manual review unresolved
final submit disabled
worker crash
```

Each error should save:

- current URL
- step name
- screenshot
- message
- suggested reason
- retry option where safe

Done Criteria

Phase 28 is complete when:

- all major edge cases have controlled error states
 - errors are saved to run logs
 - screenshots are saved for browser errors
 - retry is available where safe
 - unsafe conditions stop the run
 - UI explains the error clearly
 - tests pass
 - git status is clean
-

Phase 29 — Testing and Verification

Goal

Verify the system before production deployment.

Build Tests

Unit tests for:

- URL validation
- URL normalization
- risk classification
- question normalization
- answer mapping rules
- readiness score
- permission checks
- OTP parser
- status transitions

Integration tests for:

- Supabase connection
- candidate creation
- resume metadata
- job link creation
- run creation
- worker claim function
- manual review resolution
- final approval rules

Manual tests for:

1 Workday tenant
5 Workday tenants
20 Workday tenants
50 Workday tenants

Do not move to production before at least 5 Workday tenants work in staging.

Done Criteria

Phase 29 is complete when:

- unit tests pass
 - integration tests pass
 - lint passes
 - TypeScript build passes
 - staging dry-run works
 - at least 5 Workday tenants tested
 - known limitations are documented
 - git status is clean
-

Phase 30 — Deployment and Environment Configuration

Goal

Deploy the system safely to staging and production.

Build

Set up environments:

```
local
staging
production
```

Deploy:

- Expo web dashboard to Vercel
- Supabase database
- Supabase storage buckets
- worker to Railway / Render / Fly.io / VPS

Configure environment variables separately for:

- local
- staging
- production

Production defaults:

```
dry_run_required = true
human_approval_required = true
auto_submit_enabled = false
WORKER_CONCURRENCY = 1
```

Done Criteria

Phase 30 is complete when:

- staging deployment works
- production deployment works
- production worker starts safely
- production dry-run mode is enabled
- auto-submit is disabled
- service role key is not exposed
- storage buckets are private
- health check works
- rollback steps are documented
- git status is clean

Phase 31 — Production Pilot

Goal

Run a controlled internal pilot before scaling.

Build / Operate

Pilot sequence:

```
1 candidate
↓
1 resume
↓
1 Zoho email
↓
1 Workday link
```

↓
Dry-run only
↓
Review result
↓
5 Workday links
↓
20 Workday links
↓
50 Workday links

Track:

- success rate
- question extraction accuracy
- manual review rate
- failure reasons
- OTP success rate
- resume upload success rate
- conditional question detection rate

Done Criteria

Phase 31 is complete when:

- one full dry-run completes successfully
- five Workday links complete dry-run
- failures are understood
- manual review works
- no uncontrolled submit occurs
- operator can use system without developer help
- pilot report is created
- git status is clean

Phase 32 — V1 Release Criteria

Goal

Declare V1 ready only after safety and reliability are proven.

V1 Is Ready When

The system can:

1. create candidate
 2. upload one resume
 3. connect Zoho mailbox
 4. add Workday job links
 5. create dry-runs
 6. worker opens Workday links
 7. login or account creation works on tested tenants
 8. OTP is read from Zoho
 9. resume upload works
 10. questionnaire page is reached
 11. questions are extracted
 12. answers are mapped
 13. safe answers are filled
 14. risky answers go to manual review
 15. conditional questions are detected
 16. dry-run stops before submit
 17. human approval is required
 18. controlled submit works only after approval
 19. logs and screenshots are saved
 20. failures are understandable
-

33. Development Workflow for AI Agent

The AI agent must follow this loop for every phase:

```
Read the relevant document
↓
Implement only the current phase
↓
Run tests
↓
Run lint
↓
Run TypeScript check
↓
Show proof
↓
Show git diff summary
↓
Wait for human approval
```

↓

Move to next phase

The AI agent must not:

- skip phases
- silently change architecture
- introduce new frameworks without approval
- enable auto-submit without approval
- expose service role keys
- guess risky answers
- build multiple portals in V1
- install unnecessary agent frameworks

34. Recommended AI Agent Roles

Use Claude for:

```
PRD review
TRD review
architecture review
UI/UX reasoning
risk analysis
prompt planning
```

Use Codex for:

```
repo setup
migrations
implementation
tests
bug fixes
git diffs
build verification
deployment steps
```

Use Loop Engineering for:

```
phase planning
implementation loop
review loop
test loop
```

```
fix loop
release checklist
```

35. Recommended Prompt Pattern for Each Phase

Use this pattern when giving work to Codex:

You are building ApplyWizz Workday Dry-Run Automation Engine V1.

Read these documents first:

1. PRD
2. TRD
3. App Flow
4. ui ux
5. backend
6. implementation plan

Now implement Phase [number]: [phase name].

Strict rules:

- Implement only this phase.
- Do not jump to later phases.
- Do not change the approved architecture.
- Do not add extra frameworks.
- Do not enable auto-submit.
- Use TypeScript.
- Follow the folder structure.
- Add tests where appropriate.
- Run lint, tests, and typecheck.
- Report exact commands and results.
- Show git diff summary.
- Stop after this phase and wait for approval.

36. Final Build Order Summary

The entire V1 build order is:

1. Project Setup
2. Database Schema and Migrations
3. Authentication Flow

4. Core Layout and Navigation
5. Candidate Management
6. Resume Upload
7. Zoho Mailbox Setup
8. Job Link Management
9. Application Run Creation
10. Worker Foundation
11. Playwright Browser Base
12. Workday Page State Detection
13. Workday Login and Account Creation
14. Zoho OTP Automation
15. Resume Upload to Workday
16. Questionnaire Page Detection
17. HTML Compression and Question Extraction
18. Answer Bank and Risk Classification
19. OpenAI Answer Mapping
20. Manual Review System
21. Safe Form Filling
22. Conditional Question Handling
23. Dry-Run Completion and Readiness Score
24. Final Human Approval
25. Controlled Submit
26. Dashboard Metrics
27. UI Polish and Responsive Design
28. Error Handling and Edge Cases
29. Testing and Verification
30. Deployment and Environment Configuration
31. Production Pilot
32. V1 Release Criteria

37. Final Recommendation

Build slowly and safely.

The first major technical milestone is:

```
One candidate
+
One resume
+
One Zoho email
+
One Workday link
+
```

Worker reaches the Workday questionnaire page
+
Questions are extracted correctly

Do not worry about 90% automation on day one.

First prove:

Can we reach the question page?
Can we extract the questions?
Can we map safe answers?
Can we stop before submit?

Once these are working, automation can improve step by step.

The correct product strategy is:

Dry-run first.
Manual review second.
Controlled submit third.
Auto-submit later.